Medallion Architecture implementation using Databricks

Name: Riya Patel
NUID: 002059816

1. What changes are required in the existing model

Ans:
In the existing dimensional model, the Patient_Visit_Fact table contains only one Diag_Key. This means each visit record can store only one diagnosis. However, in real scenarios, a single patient visit may have multiple diagnoses. This creates a one-to-many relationship between a visit and its diagnoses, which the current structure cannot handle correctly.

To address this, we need to separate the diagnosis details from the fact table by introducing a new bridge table called Visit_Diagnosis_Bridge. This bridge table will store pairs of Visit_Key and Diagnosis_Key, allowing each visit to be linked to multiple diagnoses. The revised model will still maintain the existing dimensions — Patient_DIM, Doctor_DIM, Date_DIM, and Diagnosis_DIM — and the main Patient_Visit_Fact table, but with the additional bridge table to handle multiple diagnosis associations properly.

Business Requirements:

1. For a given patient, on a given date, for a doctor they visited, what are all the associated diagnosis?

Ans:
With the new model, this requirement can easily be met.
To find all diagnoses for a given patient on a given date with a specific doctor, we can join the following tables:

- Patient_Visit_Fact: provides the visit details (Patient, Doctor, Date, Visit_Key)
- Visit_Diagnosis_Bridge: links each visit to multiple diagnoses
- Diagnosis_DIM: gives the diagnosis code and description

By filtering the Patient_Key, Doctor_Key, and Date_Key, we can retrieve all corresponding Diagnosis_Code and Diagnosis_Desc values.
This works because the Visit_Diagnosis_Bridge now allows multiple diagnosis entries for one visit, which was not possible in the earlier design.

2. How many patients were diagnosed with a given diagnosis_code?

Ans:
To find how many patients were diagnosed with a particular diagnosis code, we can join:

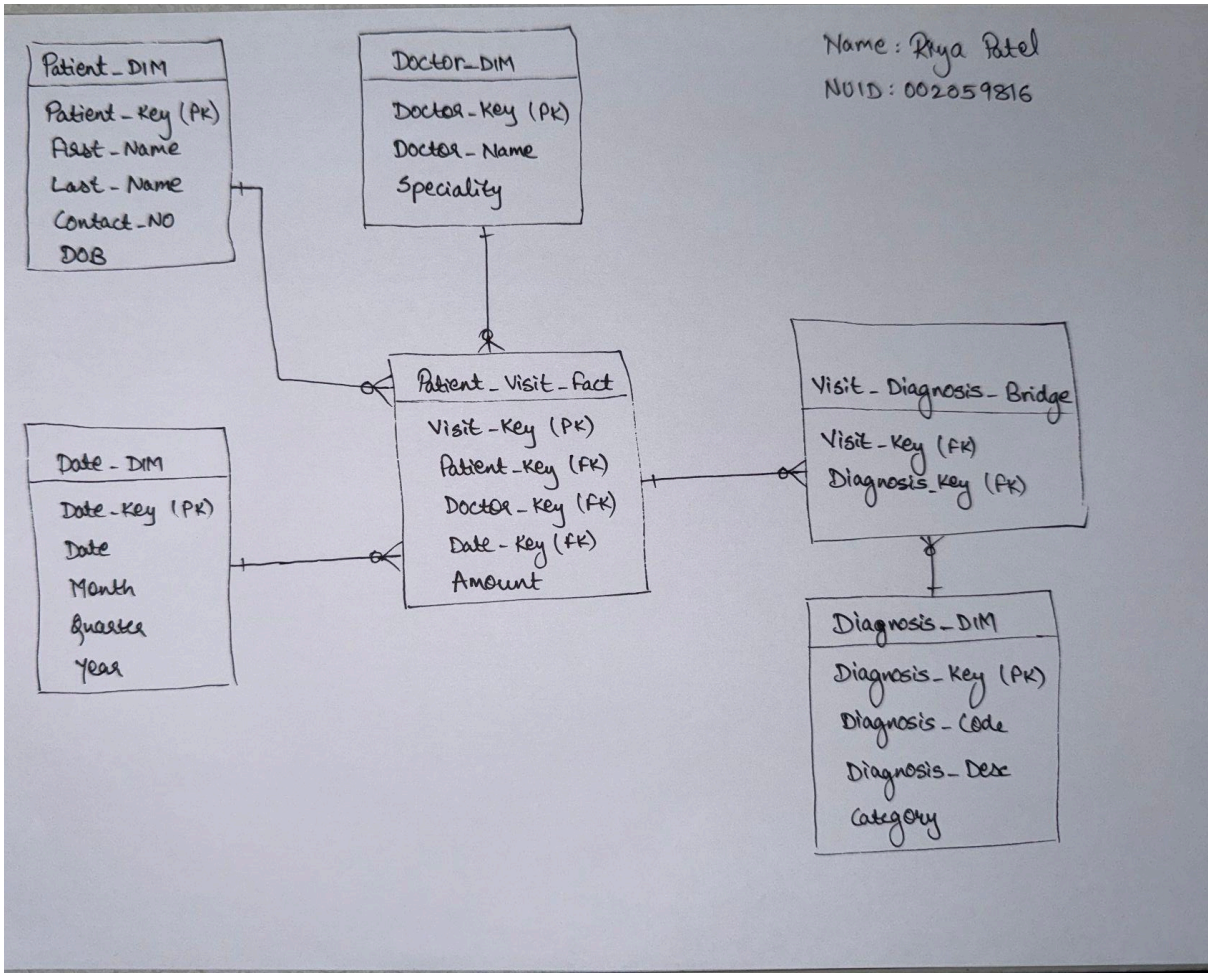
- Diagnosis_DIM (for the code)

- Visit_Diagnosis_Bridge (to find visits linked to that diagnosis)
- Patient_Visit_Fact (to get the patients who made those visits)

Then, by counting the distinct Patient_Key values, we can determine how many unique patients were diagnosed with that specific diagnosis code.
This query is now accurate and reliable because the new bridge table correctly tracks all diagnoses linked to each visit without duplication or loss of data.

Deliverables:

1. Dimension model created for Part 1



2. Document Your Approach

Ans:
The approach began by analyzing the problem with the existing model's structure. The Patient_Visit_Fact table contained a single Diag_Key, which restricted it to only one diagnosis per visit. This violated the business rule that each patient visit can have multiple diagnoses. To

correct this, we decided to redesign the model while keeping the existing fact and dimension tables intact.

We introduced a new Visit_Diagnosis_Bridge table that establishes a link between visits and diagnoses using Visit_Key and Diagnosis_Key. This bridge table enables a many-to-many relationship between patient visits and diagnosis records — one visit can have multiple diagnoses, and one diagnosis can appear in multiple visits.

The updated model now includes:

- Patient_Visit_Fact as the central fact table
- Four supporting dimension tables (Patient_DIM, Doctor_DIM, Date_DIM, and Diagnosis_DIM)
- One new bridge table (Visit_Diagnosis_Bridge).

This design maintains proper granularity (one record per visit in the fact table), improves flexibility for reporting, and fully supports both business requirements:

- Listing all diagnoses for a given patient-doctor-date visit
- Counting how many patients were diagnosed with a specific diagnosis code.

Overall, this enhanced model ensures data integrity, scalability, and more accurate analytical reporting for healthcare-related queries.